

Word Par



Created and Designed By:

- Emma McArthur
 - Github: emmamc-png
- Joel Gaikwad
 - Github: jgaikwad306
- Josiah Shehata
 - Github: jshehata7
- Karthik Chandrasekhar
 - Github: kvckarthik
- Reeves Cook
 - Github: Reeves-me

Project Description

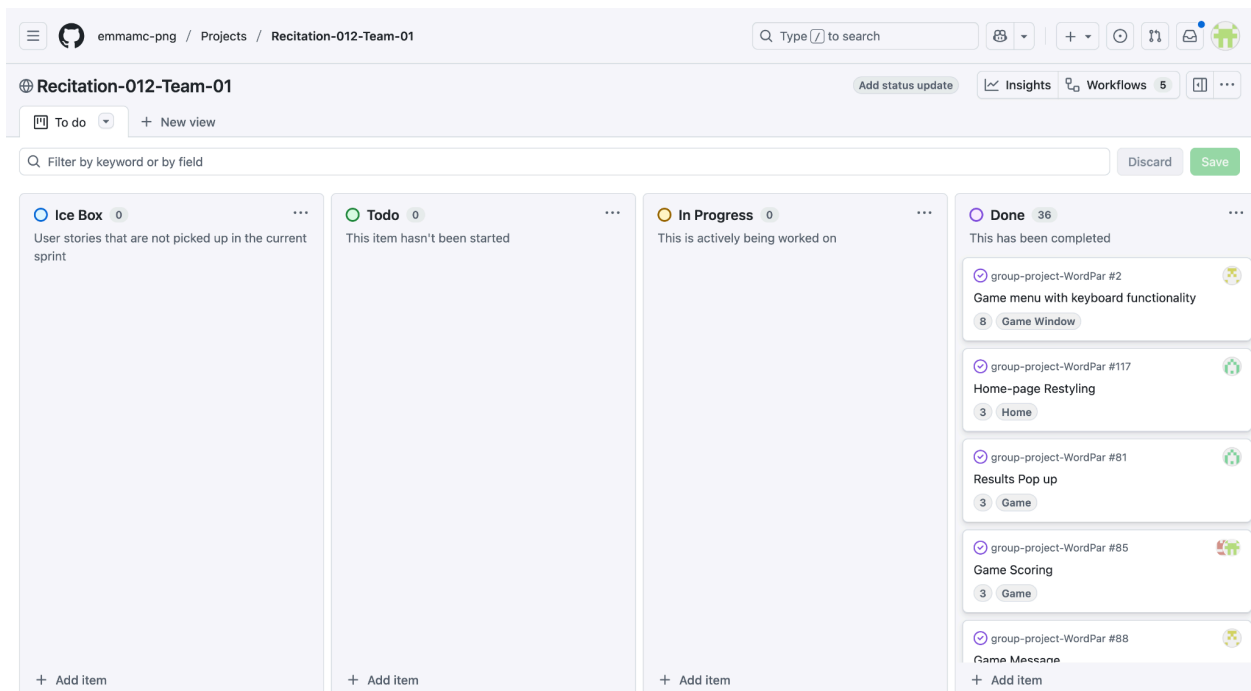
Within the game Word Par, it offers users the ability to play a word guessing game that provides a random word according to a theme of their choosing through an OpenAI API.

When the user accesses the home page after logging in to their account, they will be greeted with a leaderboard that shows the top 10 players in addition to their current ranking and total points. The leaderboard is updated every time a user refreshes the page. When a user selects play, it opens a menu to select a category before being redirected to the game and have a word selected for them to guess which will show them the amount of characters the word is. Each game starts with an amount of points available depending on the word length. Following each user guess, using the OpenAI API, it determines the points the user loses. As users go through the game, the background will change colors as well as providing them a list of their previous guesses and the percentage of "closeness" their guess is to the actual word. Every player has a maximum of ten guesses before the game ends.

Project Tracker

Our project board can be found here:

<https://github.com/users/emmamc-png/projects/4>



Video Demo

You can find our video demo here:

<https://www.youtube.com/watch?v=U4FZxU37rbo&t=1s>

VCS

You can find our git repository here:

<https://github.com/emmamc-png/group-project-WordPar>

This repository contains:

- Source code
 - Test cases
 - Video demo
 - README.md
 - Project documentation
 - Project board
-

Contributions

- Emma McArthur
 - Throughout this project, I primarily focused on the backend development of the project. I specifically worked with NodeJS, Express, and SQL to design an optimal backend for processing user requests such as for the login and registration page to change user information. Additionally, I collaborated with others for adding dynamic elements to the different webpages to ensure that users could get updated information such as the leaderboard and for the user profile modal.
- Karthik Chandrasekhar
 - For the project, my main task was researching, learning, and implementing the API sourced from OpenAI. I created JS functions and routes which allowed for word generation and semantic similarity check based on the guess word to the target word. On the game play page, when users guess a word. The word would get sent to the API routes in the back-end in a request which would send a response back to the front end with data on similarity. On the home page, I implemented the category modal that the player uses to select the target word category.
- Josiah Shehata
 - For WordPar, I primarily focused on implementing the CSS of the game page. With this, I was responsible for creating the virtual keyboard that appears and populates on the screen, caching the user's guess with an array in Javascript,

responsively displaying the guess, enforcing and providing the user with a maximum length, drawing the golf image (with a lot of shapes in CSS), and designing a dynamic background that changes based on the similarity of the user's guess.

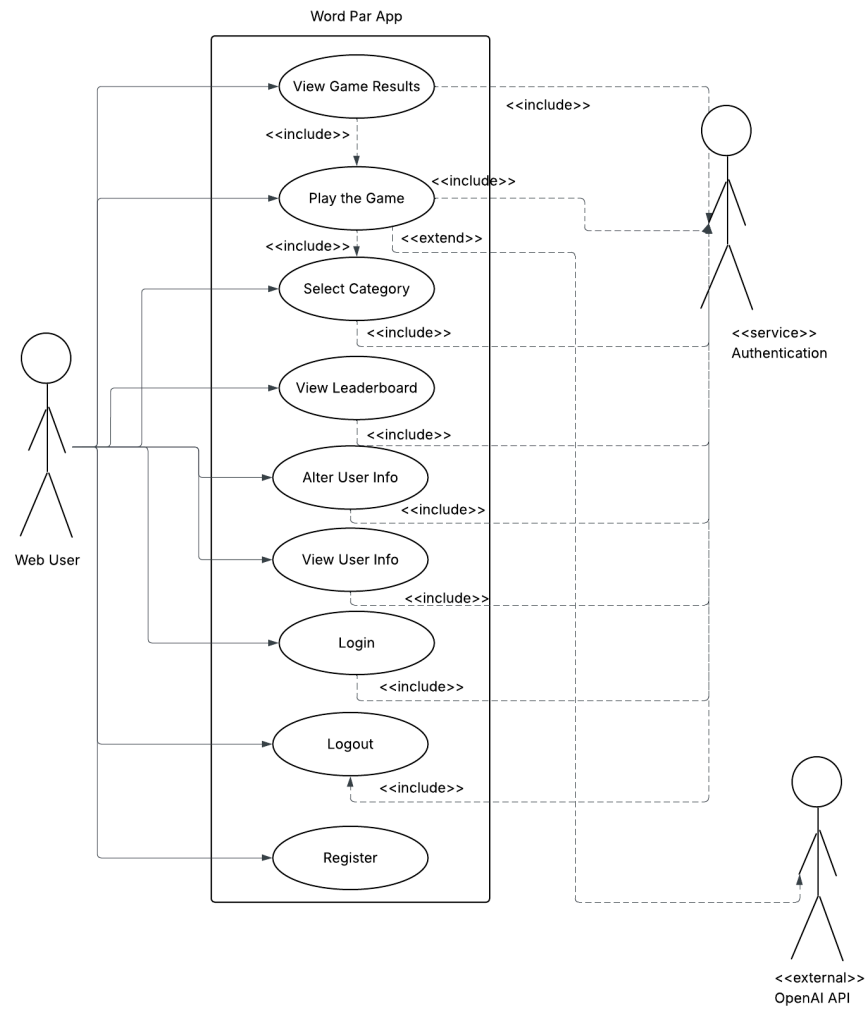
- **Joel Gaikwad**

- For WordPar I primarily worked on the database and routing it to the frontend of the website. I used PostgreSQL to create tables and then used Express to route the guesses to the database when a user submits a guess. I also worked on scoring by pulling the similarity score and putting it into an equation and displaying that score when a user submits a guess.

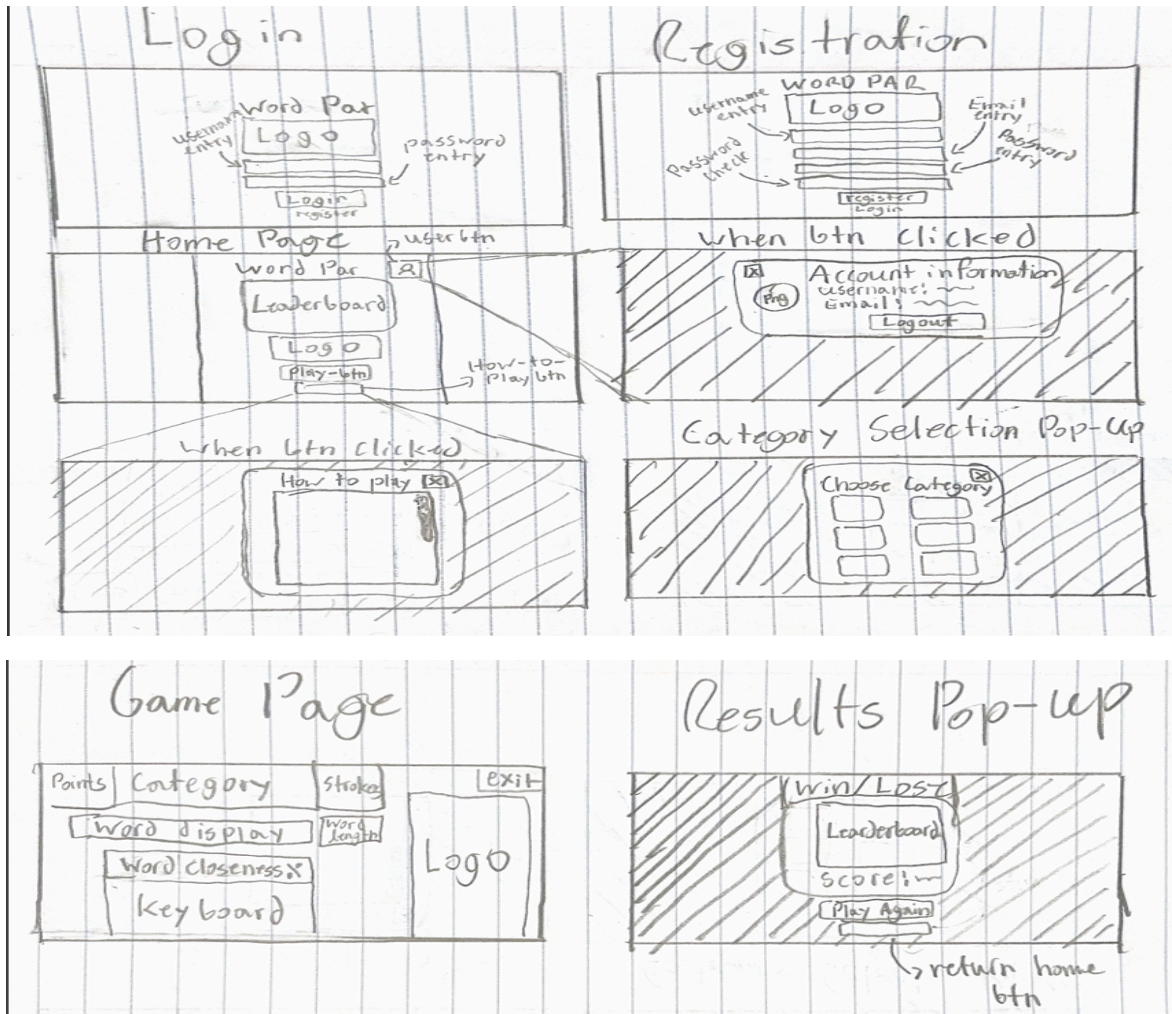
- **Reeves Cook**

- For WordPar, I focused on designing the color scheme, home, login, and registration pages. This involved primarily working on the frontend with CSS and HTML to provide a functional interface that the user can then interact with. This also includes modals like the user information dropdown and the how-to-play pop-up. I also designed and implemented the results pop-up after you have played the game, which displays the leaderboard and specific text depending on whether you win or lose through the use of JavaScript.

Use Case Diagram



Wireframes



Test Results

Testers: The people who will be using this app who enjoy golf and word games including a variety of ages from college students to elderly. The platform has to be accommodating of all users utilizing the platform. For this testing event, we utilized a diverse group of college students.

1. Users should be able to enter a guess with the keyboard

To test this, we will simulate a user entering the word "ADIEU." We will enter the game page, via the localhost environment, and type this word using the keyboard only. Upon typing this word, we will hit the "Enter" key. We will then examine the word that appears on the screen. If the word that appears is "ADIEU", then our test case was successful.

Observations: Through simulating this test case with multiple test users, we noticed a variety of interactions the users had. All of the users were confused when it came to the keyboard being on

the screen and were under the impression they could not use their device's keyboard. Since we had not provided them information about this, I took the initiative to clarify this for them. Once they knew, they preferred utilizing their device's keyboard instead. Overall, when using this use case, all the users completed it without issue and the game acted accordingly with the background changing based on similarity. No changes were made to the application at this time as we think of a new method of highlighting to the player that they can use both their device's keyboard and the one on the screen.

2. Users should be able to log into their account via their correct username and password.

To test this, we will use the localhost environment, and simulate a user logging. We will first navigate to the website, where we should be prompted by a login page. Since we are creating a new account, we will then navigate to the registration page by clicking on *Register*. Here, we will make a new user with the following data:

Username: Austin

Email: 123austin@aol.com

Password: password:)

Retype Password: password:)

We will then press the *Register* button, and the site should redirect us to the login page. From here, we will input our newly created user credentials into the page:

Username: Austin

Password: password:)

Upon hitting the *Login* button, we will see what the page does. If we are redirected into the main home page of Word Par, our test case is successful.

Observations: When provided with the test case data and with the database being cleared and restarted between users, the users were easily able to register and login. After having the information given to them and being placed on the login screen, they instinctively knew where the registration page would be located and clicked on the button. Upon doing so, the instructions were clear and they found the email RegEx and password match RegEx to be extremely helpful so they knew if they were inputting the wrong information. Upon completion of the registration, one of the users wished that the login page had automatically input the information they had just registered with, but the others had no suggestions. Finally they submitted their login information and were sent to the home page, making these test cases successful. All users found it simple and their actions/behaviors were consistent with the use case. It was easy to navigate and simple enough to figure out what to do without instructions.

3. Users should be able to view the top ten scores on the leaderboard

To test this, we will use the localhost environment, and simulate a user viewing the leaderboard

We will use the account created in the last user test. To start off, we will navigate to the login in page, and input the following information:

Username: Austin

Password: password:)

Upon doing this, we should be navigated to the main page, and we will view the results. If the leaderboard is displayed on the page, our test case was successful.

Observations: Upon being provided with the user information, all test users instinctively knew how to login. Once logging in the users were all directed to the home page with the leaderboard and immediately went to scroll through it to see how many users you could view. By doing this, they immediately knew the top 10 users were shown on the leaderboard in terms of score. They also checked their current ranking; however, since none of them had played yet, they were unranked. One piece of feedback received from two of the users was that they wished it was a bit bigger and also had more players available to see, such as maybe a top 25 or top 100. At this time, because we were trying to keep the leaderboard at a comfortable size to read without users having to scroll a while and having difficulty using it, we decided to keep only the top 10 players. However, in the future, if we change the home screen design, we would implement a larger leaderboard with more information.

4. Users should be able to change their password through the profile modal

To test this, we will use the localhost environment, and simulate a user changing their password through their profile modal. To start off, we will navigate to the login in page, and input the following information:

Username: Austin

Password: password:)

Upon completion of this task, we should be navigated to the home page, and we will click on the user profile icon in the upper right corner of the main game screen. Upon clicking on the profile, a modal pop-up should appear on the screen with a drop down selection on the left. We will select “Change Password” from the options and two boxes will appear: current password and new password. We will enter into these boxes:

Current Password: password:)

New Password: pass:)

Upon clicking the submit button, we will click the “logout” button on the profile modal and be sent back to the login page. Upon completion of this, we will enter the following information into the login page:

Username: Austin

Password: pass:)

Upon doing this, we should be navigated to the home page. If the home page is displayed on the page, our test case was successful.

Observations: Overall, the users were easily able to navigate through these steps. Some inconsistencies I noticed were that sometimes the users found it difficult to figure out where the profile picture was located because it's generally in the far right upper corner. Additionally, the users found it to immediately mess with the profile picture instead of anything else because they did not know there was one previously. In the future, we would implement another place for this user profile picture to be utilized, whether it's in the leaderboard or somewhere else. Additionally, once they finally noticed that they could change their user information, they found it quite simple, but they felt like it was unsecure. Although we did make them re-enter their current password as a safety precaution, one of the users who specializes in cybersecurity suggested using a form sent to the user's email instead. However, this would have required utilizing an additional API that was too late to implement, but they did overall say it should not be a main priority. Overall, the user behavior was consistent, but immediately after changing the password, 75% of the testers immediately forgot what they changed their password to. However, this is something that is difficult for us to control as it's up to the user to remember their information.

Deployment

You can find our application deployed on Render here:

<https://group-project-wordpar-1.onrender.com/>